

第六章 管理 SELinux

普通文件权限允许访问，并不代表系统一定会允许这次访问。SELinux 会在传统权限检查之后，再根据进程和目标对象的安全类型、请求的行为以及当前策略作出判断。本章围绕一个使用自定义目录和非标准端口的 Web 服务展开：先从“服务为什么失败”建立证据，再学习怎样配置持久规则，最后把文件类型、端口类型、Boolean 和 AVC 日志串成一条能够用于考试的处理路径。

SELinux 中最重要的不是先背命令，而是先分清“谁在访问、访问什么、请求什么行为、策略为什么允许或拒绝”。下面的概念和操作语义先建立本章的最小地图。

[概念] 自主访问控制（Discretionary Access Control, DAC）与强制访问控制（Mandatory Access Control, MAC）：用户、组和权限位属于传统 DAC；SELinux 在 DAC 允许之后继续执行强制策略判断，因此普通权限正确仍可能被拒绝。

[概念] 安全上下文、类型（type）与进程域（domain）：文件和进程都带有安全上下文。RHCSA 常见服务问题主要关注第三段 type；进程的 type 常称为 domain，例如 `httpd_t`，文件类型则描述对象用途，例如 `httpd_sys_content_t`。

[概念] 文件上下文规则、端口类型与 Boolean：文件上下文规则定义某类路径应具有持久类型；端口类型定义某类服务可以使用哪些端口；Boolean 是策略预留的行为开关。三者分别处理对象、端口和服务行为，不能互相替代。

[概念] AVC（Access Vector Cache）拒绝记录：它说明哪个进程域对哪个对象请求了什么行为并被策略拒绝，是从现象进入 SELinux 证据链的主要入口。

[操作语义] `getenforce` / `sestatus`：查看当前执行模式以及当前状态与持久配置是否一致。

[操作语义] `semanage fcontext` + `restorecon`：前者建立路径的持久类型规则，后者让现有文件按照规则恢复到预期标签。

[操作语义] `semanage port` / `setsebool -P`：分别管理服务端口类型和持久 Boolean，解决的是不同策略入口。

[操作语义] `ausearch -m AVC -ts recent`：查询近期拒绝证据，本质是缩小到当前失败操作，而不是自动生成修复答案。

[知识专题] 普通权限正确，为什么访问仍会被拒绝

设想一台服务器已经安装并配置了 `httpd`。站点内容放在 `/srv/site`，服务计划监听 TCP 82。目录对 Web 进程具备基本的 Unix 读取权限，防火墙也允许访问，但服务仍可能无法启动，或者启动后访问返回 `403 Forbidden`。

这种现象不能直接归因于 SELinux。正确的起点仍然是查看证据：服务是否在运行，配置是否通过语法检查，目标端口是否真正监听，普通权限是否允许访问，防火墙是否放行。只有这些信息不能解释现象，或者日志中出现 AVC 拒绝时，才有理由把注意力进一步收缩到 SELinux。

传统的自主访问控制（Discretionary Access Control, DAC）通常根据用户、组和权限位判断访问。SELinux 在 DAC 允许之后继续检查强制访问控制规则。也就是说，普通权限拒绝时，SELinux 不会替它放行；普通权限允许时，SELinux 仍可能拒绝。

当我们查看文件或进程的安全上下文时，会看到类似下面的结构：

```
system_u:object_r:httpd_sys_content_t:s0
```

它依次包含 SELinux 用户、角色、类型和安全级别。在 RHCSA 常见的服务问题中，最需要关注的通常是第三段 `type`。对进程而言，这个类型也常称为 domain，例如 `httpd_t`；对文件而言，它描述对象用途，例如 `httpd_sys_content_t`。策略会结合进程域、目标对象类型和请求的行为，判断是否允许这次访问。



这里最重要的不是背完整的四段字符串，而是能够识别：**当前是哪个进程域，正在对哪种对象类型请求什么行为**。这三个问题也是后面阅读 AVC 记录时最有价值的线索。

[操作专题] 识别当前模式与持久配置

SELinux 可以处于 `Enforcing`、`Permissive` 或 `Disabled`。在 `Enforcing` 模式下，策略会真正拒绝不允许的访问；在 `Permissive` 模式下，系统仍然产生标签并记录本应发生的拒绝，但不阻止操作；`Disabled` 表示 SELinux 基础设施没有加载，也不会正常维护标签。

先用下面的命令确认当前模式：

```
getenforce
```

需要同时观察当前状态和配置文件中的启动模式时，`sestatus` 更直接：

```
sestatus
```

代表性输出中，下面两行回答的是两个不同问题：

```
Current mode:                enforcing
Mode from config file:       enforcing
```

前者说明当前内核正在怎样执行策略，后者说明系统下次按配置启动时计划进入什么模式。两者相同才说明当前状态与持久配置一致。

`setenforce` 只能在已启用 SELinux 的系统中临时切换 Enforcing 与 Permissive：

```
setenforce 0
setenforce 1
```

`setenforce 0` 可以帮助验证 SELinux 是否参与了故障，但现象在 Permissive 下消失，只能说明 SELinux 可能阻止了某个行为，**不能说明问题已经修复**。下一步仍要重新触发失败操作，查看 AVC，并找到文件标签、端口类型或 Boolean 等标准配置入口。

持久模式由 `/etc/selinux/config` 管理。例如题目要求系统保持 Enforcing，应确认：

```
SELINUX=enforcing
SELINUXTYPE=targeted
```

修改配置文件不会立即改变当前模式，`setenforce` 也不会修改该文件。因此，考试题若同时要求“当前处于 Enforcing”并且“重启后仍保持 Enforcing”，就必须分别验证这两个状态。

关键提醒

不要通过把 SELinux 切换为 Permissive 或 Disabled 来完成服务题。降级保护只能作为有限的诊断证据，最终解法必须修正具体的策略条件。

[验证]

```
getenforce
sestatus
grep -E '^SELINUX(=|TYPE=)' /etc/selinux/config
```

[知识专题] 从一个失败的 Web 服务开始取证

主场景中的 `httpd` 已经配置为使用 `/srv/site` 作为站点目录，并监听 TCP 82。开始修改 SELinux 之前，先确认其他层次提供了什么证据。

```
httpd -t
systemctl status httpd
ss -lntp | grep ':82'
namei -l /srv/site/index.html
firewall-cmd --list-ports
```

这些命令不是要在 SELinux 章节中重讲 `systemd`、`Apache`、文件权限和 `firewalld`，而是为了避免跳过调查直接猜答案。`httpd -t` 检查配置语法，`systemctl status` 说明服务当前状态，`ss` 说明端口是否实际监听，`namei -l` 可以帮助查看路径各层目录的普通权限，防火墙查询则说明网络过滤层是否允许该端口。

假设配置语法正确，但 `httpd` 启动时报告无法绑定 TCP 82；或者端口问题处理后，服务已经运行，本机访问 `/srv/site/index.html` 仍然返回 403。此时应重新触发失败操作，并查询近期 SELinux 拒绝：

```
ausearch -m AVC -ts recent -i
```

代表性的端口绑定拒绝可能包含下面的稳定字段：

```
avc: denied { name_bind }
comm="httpd"
scontext=...:httpd_t:s0
tcontext=...:port_t:s0
tclass=tcp_socket
```

阅读时不必被 PID、时间戳和完整上下文淹没。先提炼四件事：`httpd` 发起了访问；被拒绝的行为是 `name_bind`；源进程类型是 `httpd_t`；目标是一个尚未被策略视为 Web 端口的 TCP socket。若拒绝的是文件读取，则重点会转向目标路径和目标文件类型。

如果查询不到记录，先确认刚刚是否重新触发了失败操作、时间范围是否合适、`auditd` 是否运行。必要时再查看 `journalctl -t setroubleshoot` 等替代入口，但不要把排错变成依次尝试一长串日志命令。

[操作专题] 当前标签与持久文件上下文规则

将内容从默认目录移动到 `/srv/site` 后，文件往往继承 `/srv` 的类型，而不是 Web 服务能够读取的 `httpd_sys_content_t`。先查看当前标签：

```
ls -Zd /srv/site
ls -LZ /srv/site
```

`ls -Z` 只能说明对象此刻携带什么标签。即使执行下面的命令后看到类型正确，也不能证明配置具有持久性：

```
chcon -t httpd_sys_content_t /srv/site/index.html
```

`chcon` 直接改变当前标签，适合用来理解“标签改变会影响访问”或进行短暂验证，但它没有建立路径的持久规则。以后执行 `restorecon`、重新标记文件系统或重新创建对象时，标签可能回到策略认为该路径应该具有的值。

稳定的做法是先登记路径规则：

```
semanage fcontext -a \
-t httpd_sys_content_t \
'/srv/site(/.*)?'
```

`/srv/site(/.*)?` 同时覆盖目录本身和其下的文件、子目录。这里不需要学习完整正则语法，只要能解释：`/srv/site` 是目录本身，后面的可选部分代表“斜杠以及其后的任意内容”。单引号把表达式原样交给 `semanage`，避免 Shell 对特殊字符产生干扰。

`semanage fcontext` 只记录“这类路径应是什么类型”，不会自动修改现有文件。接着使用 `restorecon` 把规则应用到当前对象：

```
restorecon -Rv /srv/site
```

如果 `semanage` 命令不存在，不要在每个 SELinux 工具旁都展开软件包清单。只需记住通用恢复思路：先检查拼写，再通过仓库反查命令来源，例如：

```
dnf provides '*/semanage'
```

确认并安装对应的软件包后继续。后续其他工具缺失时沿用同一思路。

[验证] 文件上下文配置至少要回答三个不同问题：持久规则是否存在，策略认为路径应该是什么标签，文件当前实际是什么标签。

```
semanage fcontext -l | grep -F '/srv/site'
matchpathcon /srv/site/index.html
ls -lZ /srv/site/index.html
```

若预期标签与当前标签不一致，再执行 `restorecon` 并重新检查。三者一致，才有理由认为当前状态和持久规则形成闭环。

[常见错误] 只执行 `chcon`，看到 `ls -Z` 显示为 `httpd_sys_content_t` 就停止。这里验证到的只是当前标签，尚未证明路径规则存在。

本专题的推荐路径可以重新串成：

```
semanage fcontext -l | grep -F '/srv/site'

semanage fcontext -a \
  -t httpd_sys_content_t \
  '/srv/site(/.*)?'

restorecon -Rv /srv/site

matchpathcon /srv/site/index.html
ls -lZ /srv/site/index.html
```

如果查询发现同一表达式已经存在，不应机械重复 `-a`。先确认原规则的类型，再根据目标决定是否使用修改操作：

```
semanage fcontext -m \
  -t httpd_sys_content_t \
  '/srv/site(/.*)?'
```

[知识专题] 文件类型表达服务可以做什么

静态站点内容通常只需要由 `httpd` 读取，因此使用 `httpd_sys_content_t`。若业务确实要求 Web 服务写入某个目录，例如上传目录或应用生成内容，可以考虑为那一小块目录使用 `httpd_sys_rw_content_t`。

这不是简单的类型名称替换。它表达的是不同的行为边界：一个目录只需读取，就不应因为配置方便而授予服务写入能力；只有明确需要写入的对象，才使用可写类型。例如 `/srv/site` 作为静态站点根目录，而 `/srv/site/uploads` 用于上传，可以分别登记：

```
semanage fcontext -a \
  -t httpd_sys_content_t \
  '/srv/site(/.*)?'

semanage fcontext -a \
  -t httpd_sys_rw_content_t \
  '/srv/site/uploads(/.*)?'

restorecon -Rv /srv/site
```

即使 SELinux 类型允许写入，普通 Unix 所有者、组和权限位仍然必须允许相应进程写入；反过来也一样，普通权限允许不代表 SELinux 一定允许。排错时应明确自己正在检查哪一层，而不是看到“Permission denied”就只反复执行 `chmod`。

[操作专题] 允许服务使用非标准端口

Apache 配置中的 `Listen 82`、防火墙中的 `82/tcp` 和 SELinux 中的端口类型是三个独立状态。防火墙放行只能说明网络过滤层允许流量，不能证明 `httpd_t` 进程域被策略允许绑定这个端口。

修改前先查看 `http_port_t` 当前包含哪些 TCP 端口：

```
semanage port -l | grep -w http_port_t
```

若 TCP 82 尚未登记，添加映射：

```
semanage port -a \
  -t http_port_t \
  -p tcp \
  82
```

这里 `-a` 表示新增记录，`-t` 指定 SELinux 类型，`-p` 指定协议。端口映射区分 TCP 与 UDP，不能因为端口号相同就省略协议判断。

如果命令提示端口已经定义，不要反复执行同一条新增命令。先查目标端口当前属于什么类型：

```
semanage port -l | grep -w '82'
```

确认现有映射与任务目标冲突时，再考虑修改：

```
semanage port -m \  
-t http_port_t \  
-p tcp \  
82
```

[验证] 端口规则存在只能证明 SELinux 允许该类服务使用这个端口。完整链路还应分别确认 Apache 配置、进程实际监听、防火墙和最终访问：

```
grep -nE '^Listen[[:space:]]+82$' /etc/httpd/conf/httpd.conf  
semanage port -l | grep -w http_port_t  
systemctl is-active httpd  
ss -lntp | grep ':82'  
firewall-cmd --list-ports  
curl -I http://localhost:82/
```

此时主场景中的端口限制已经处理。如果站点仍返回 403，而文件上下文尚未修复，就说明端口和目录是两条独立的策略路径；局部成功不能替代综合验收。

[操作专题] Boolean：从业务行为找到已有策略入口

Boolean 用来启用或关闭策略预先提供的可选行为。它不是“SELinux 出错时随便试几个开关”，而是当业务确实需要某类行为时，查找已有策略是否提供了最小入口。

例如，若 Web 应用需要主动连接 MariaDB，可以先搜索与 `httpd` 相关的 Boolean：

```
getsebool -a | grep '^httpd_'  
semanage boolean -l | grep httpd
```

找到与业务语义匹配的 `httpd_can_network_connect_db` 后，再持久启用：

```
setsebool -P httpd_can_network_connect_db on
```

不带 `-P` 的 `setsebool` 只改变当前值；`-P` 会将修改写入持久策略配置，执行时可能需要一些时间。修改前后都应使用 `getsebool` 验证：

```
getsebool httpd_can_network_connect_db
```

Boolean 不能替代文件标签、端口类型、普通权限和服务配置。若开关已经启用但访问仍失败，继续根据 AVC 中的对象、行为和上下文调查，而不是盲目打开更多权限。

[诊断专题] 从 AVC 记录推进到下一条证据

AVC (Access Vector Cache) 记录的价值在于解释“谁对什么对象做什么时被拒绝”。它不是一条可以直接复制的修复命令。看到拒绝后，应先判断它属于哪一种常见偏差：对象标签不正确，服务使用了非标准端口，业务需要某个已有 Boolean，还是服务本身配置错误。

`audit2why` 可以辅助解释审计拒绝，但它提供的是分析线索，不是绝对结论。`audit2allow` 能依据拒绝生成候选策略规则，却不应成为第一选择。RHEL 官方文档明确建议先检查标签问题，以及服务是否改变了默认目录、端口或行为而没有同步告诉 SELinux。

关键提醒

不要把“工具建议生成自定义策略”理解为“标准修复就是运行 `audit2allow -M`”。优先寻找现有的文件类型、端口类型和 Boolean。只有明确确认标准策略无法表达合法需求时，才进入自定义策略范围；这不属于本章核心操作路径。

主场景完整走过之后，可以把诊断思路收束为一张证据推进图：



这张图不是要求每次机械地从第一步执行所有命令。若题目已经提供了可靠证据，可以从已知状态继续；但每一个结论都应能回答“我凭什么这样判断”。

[经典任务] 在 Enforcing 模式下恢复非标准 Web 服务

一台 RHEL 9 服务器已经安装 `httpd`，并完成以下业务配置：

- Apache 使用 `/srv/exam-site` 作为站点目录；
- `/srv/exam-site/index.html` 已存在，内容不得删除或改写；
- Apache 配置为监听 TCP 82；
- `firewalld` 正在运行；
- 当前访问失败，系统可能同时存在多个配置偏差。

请将系统配置到下面的目标终态：

1. SELinux 当前处于 `Enforcing`，重启后的配置也保持 `Enforcing`；
2. `httpd` 当前运行，并配置为开机自动启动；
3. `httpd` 可以在 TCP 82 上监听；
4. 客户端能够通过 `http://servera:82/` 读取现有首页；
5. `/srv/exam-site` 的 SELinux 文件上下文配置是持久的，执行 `restorecon` 后仍保持正确；
6. 不通过关闭 SELinux、删除原文件或安装自定义策略模块绕过问题。

验收时至少应能证明： 当前模式与持久模式一致，文件上下文规则与当前标签一致，TCP 82 属于正确的 SELinux 端口类型，服务当前状态与开机启动状态正确，端口实际监听，防火墙允许访问，最终 HTTP 请求成功。

卡住时再想一想

当前标签正确，是否就能证明路径规则已经持久存在？

防火墙允许 TCP 82，是否能证明 SELinux 允许 `httpd` 绑定该端口？

服务显示 `enabled`，是否能证明它现在正在运行？

请先根据验收要求独立完成。参考分析与推荐实现从下一页开始。

[参考解答] 参考分析与推荐实现

先把题目拆成几个彼此独立的状态：SELinux 模式、文件上下文、端口类型、服务当前状态、开机启动、防火墙和最终业务访问。任何一个局部状态正确，都不能单独证明整个任务完成。

[分析思路] 先查询而不是直接修改。下面这组命令用于建立初始证据：

```
getenforce
sestatus
httpd -t
systemctl status httpd
systemctl is-enabled httpd
ss -lntp | grep ':82'
semanage port -l | grep -w http_port_t
semanage fcontext -l | grep -F '/srv/exam-site'
matchpathcon /srv/exam-site/index.html
ls -lZ /srv/exam-site/index.html
firewall-cmd --list-ports
ausearch -m AVC -ts recent -i
```

如果 `httpd` 因 TCP 82 的 `name_bind` 拒绝而无法启动，先处理端口类型。目标端口尚未登记时：

```
semanage port -a \
  -t http_port_t \
  -p tcp \
  82
```

若端口已存在，先确认现有类型，再根据实际状态决定是否使用 `-m`，不要反复执行 `-a`。

接着为自定义站点目录建立持久规则，并应用到现有文件：

```
semanage fcontext -a \
  -t httpd_sys_content_t \
  '/srv/exam-site(/.*)?'
restorecon -Rv /srv/exam-site
```

这里若只执行 `chcon`，短时间内可能看到页面恢复，但持久规则仍然缺失。因此参考路径直接从 `semanage fcontext` 建立规则，不把临时标签当作最终配置。

确认 `/etc/selinux/config` 中的模式为：

```
SELINUX=enforcing
SELINUXTYPE=targeted
```

若当前模式是 Permissive，在修正具体策略条件后切回 Enforcing：

```
setenforce 1
```

然后确保服务当前运行并配置开机启动：

```
systemctl enable --now httpd
```

若 firewalld 尚未允许 TCP 82，添加持久规则并重新加载：

```
firewall-cmd --permanent --add-port=82/tcp
firewall-cmd --reload
```

[验证] 验证不应只看某条配置命令是否返回成功。依次证明各个目标状态：

```
getenforce
sestatus

grep -E '^SELINUX(=|TYPE=)' /etc/selinux/config

semanage fcontext -l | grep -F '/srv/exam-site'
matchpathcon /srv/exam-site/index.html
ls -lZ /srv/exam-site/index.html

semanage port -l | grep -w http_port_t

systemctl is-active httpd
systemctl is-enabled httpd
ss -lntp | grep ':82'

firewall-cmd --list-ports
curl -I http://localhost:82/
```

理想情况下，当前模式和配置模式均为 Enforcing；路径规则指向 `httpd_sys_content_t`，当前标签与预期一致；`http_port_t` 包含 TCP 82；服务同时为 `active` 和 `enabled`；端口实际监听；防火墙列出 `82/tcp`；HTTP 请求返回成功状态。

最后重新执行：

```
restorecon -Rv /srv/exam-site
```

再检查标签和页面访问。这样可以验证站点不是依赖一次临时 `chcon` 才能工作。

本任务中最容易出现的三个误判是：看到 `enabled` 就认为服务正在运行；看到防火墙放行就认为端口链路已完整；看到 `ls -Z` 当前标签正确就认为文件上下文已经持久。它们都把一个局部证据错误地扩大成了最终结论。

[本章收束] 回到本章的问题

SELinux 故障不应从“关闭保护”开始，而应从证据开始。普通权限、服务状态、防火墙和 SELinux 各自回答不同问题；在确认 SELinux 参与拒绝后，再从进程域、目标对象和被拒绝行为中判断应检查文件类型、端口类型还是 Boolean。

文件上下文题最重要的关系是：`semanage fcontext` 建立持久路径规则，`restorecon` 按规则调整当前标签，`ls -Z` 只观察当前状态，`matchpathcon` 帮助比较预期与实际。非标准端口题则要把 Apache 配置、实际监听、防火墙和 `http_port_t` 四个状态分开验证。Boolean 只有在业务确实需要某类可选行为时才启用，并通过 `-P` 明确持久性。

学完本章后，你应能在不关闭 SELinux 的前提下，处理自定义 Web 目录和非标准端口，读懂一段精简的 AVC 拒绝，并用“查询当前状态—选择最小持久修改—重新触发—综合验证”的方式完成任务。